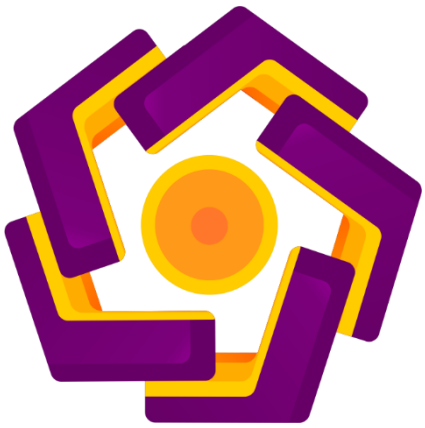
TUGAS 2 IT PLATFORM AND INFRASTRUCTURE

LAPORAN KOMPARASI PERFORMA IMPLEMENTASI PIPELINE PEMPROSESAN BATCH HADOOP MAPREDUCE VS APACHE SPARK PADA NASA HTTP SERVER LOG



Disusun oleh:

Muhammad Abiyyu Rahman (25.51.1738)

Muhammad Gufron Hawaly (25.51.1735)

PROGRAM MAGISTER

FAKULTAS ILMU KOMPUTER

PROGRAM STUDI TEKNIK INFORMATIKA

KONSENTRASI DIGITAL TRANSFORMATION INTELLIGENCE

UNIVERSITAS AMIKOM YOGYAKARTA

YOGYAKARTA

2025/2026

1. Pendahuluan

Laporan ini menyajikan perbandingan performa antara dua teknologi pemrosesan data besar yang populer: Hadoop MapReduce dan Apache Spark. Eksperimen dilakukan menggunakan dataset *NASA HTTP Server Access Log* untuk menjalankan lima tugas analisis data yang berbeda. Tujuan utama adalah mengevaluasi efisiensi waktu eksekusi, penggunaan memori, dan kemudahan implementasi dari masing-masing teknologi.

Berikut link GitHub (Implementasi Kode) : https://github.com/iuaiyuuu/batch-pipeline-hadoop-spark/tree/main

2. Deskripsi Dataset

Dataset yang digunakan adalah log akses HTTP server milik NASA Kennedy Space Center untuk periode Juli hingga Agustus 1995. Dataset ini memiliki karakteristik sebagai berikut:

- **Total Records:** 3.461.612 baris
- **Format:** Common Log Format (CLF)
- **Ukuran Data:** Sekitar 300 MB
- **Struktur Data:** host - - [date] "METHOD /endpoint HTTP/version" status bytes

3. Metode Analisis Task

Lima tugas analisis dijalankan secara identik pada tiga pipeline (Hadoop MapReduce, Spark RDD API, dan Spark DataFrame API):

1. **T1 (Top Hosts):** Menemukan 10 host/IP dengan request terbanyak.
2. **T2 (Status Distribution):** Menghitung distribusi HTTP status code.
3. **T3 (Daily Traffic):** Menghitung total request per hari.
4. **T4 (Top Endpoints):** Menemukan 10 URL endpoint paling sering diakses.
5. **T5 (Error Rate):** Menghitung persentase request dengan status 4xx dan 5xx.

4. Hasil Eksperimen

4.1 Ringkasan Performa Keseluruhan

Metrik	MapReduce	Spark RDD	Spark DataFrame
Total Execution Time	433.14 detik	78.08 detik	8.65 detik
Average Memory Peak	5.46 MB	0.20 MB	0.13 MB
Speedup vs MapReduce	1.00x	5.55x	50.05x

4.2 Waktu Eksekusi per Task (Detik)

Tugas	MapReduce	Spark RDD	Spark DataFrame
T1: Top Hosts	89.53	18.80	3.78
T2: Status Distribution	93.84	14.42	1.34
T3: Daily Traffic	83.18	13.07	1.20
T4: Top Endpoints	83.91	14.26	1.35
T5: Error Rate	82.67	17.53	0.98

5. Analisis dan Pembahasan

5.1 Waktu Eksekusi

MapReduce adalah yang paling lambat karena bersifat *disk-based*. Setiap tahap operasi menulis data sementara ke disk, yang menciptakan bottleneck I/O yang signifikan. Sebaliknya, Spark RDD melakukan pemrosesan di memori (RAM), membuatnya 5.5x lebih cepat. Spark DataFrame memberikan performa terbaik (50x lebih cepat) berkat *Catalyst Optimizer* yang mengoptimalkan rencana eksekusi secara otomatis.

5.2 Penggunaan Memori

Spark menunjukkan efisiensi yang jauh lebih tinggi dalam manajemen memori di sisi driver dibandingkan MapReduce. Melalui optimasi DataFrame, penggunaan memori dapat ditekan hingga tingkat minimal karena distribusi beban kerja ke worker nodes yang lebih efektif.

5.3 Kemudahan Implementasi

Spark DataFrame menawarkan sintaks yang paling ringkas dan intuitif (mirip SQL). MapReduce memerlukan kode yang paling panjang (verbose) karena mengharuskan definisi kelas mapper dan reducer secara manual untuk setiap tugas.

6. Kesimpulan

Berdasarkan eksperimen ini, **Apache Spark dengan DataFrame API** adalah solusi yang paling direkomendasikan untuk pemrosesan batch data log. Teknologi ini memberikan keunggulan mutlak dalam hal kecepatan, efisiensi sumber daya, dan kemudahan pengembangan dibandingkan Hadoop MapReduce tradisional.

Referensi Jurnal

1. **[1]** Abdalla, H. B., Kumar, Y., Zhao, Y., and Tosi, D. (2025). A Comprehensive Survey of MapReduce Models for Processing Big Data. *Big Data and Cognitive Computing*, 9(4), 77. DOI: https://doi.org/10.3390/bdcc9040077
2. **[2]** Gabdullin, M. T., et al. (2024). Comparative Analysis of Hadoop and Spark Performance for Real-time Big Data Smart Platforms Utilizing IoT Technology in Electrical Facilities. *Journal of Electrical Engineering and Technology*, 19, 4595-4606. DOI: https://doi.org/10.1007/s42835-024-01937-1
3. **[3]** Empirical Evaluation of Big Data Stacks: Performance and Design Analysis of Hadoop, Modern, and Cloud Architectures. (2026). *Big Data and Cognitive Computing*, 10(1), 7. DOI: https://doi.org/10.3390/bdcc10010007
4. **[4]** Theodorakopoulos, L., Karras, A., and Krimpas, G. A. (2025). Optimizing Apache Spark MLlib: Predictive Performance of Large-Scale Models for Big Data Analytics. *Algorithms*, 18(2), 74. DOI: https://doi.org/10.3390/a18020074